Kubeadm Installation Guide

This guide outlines the steps needed to set up a Kubernetes cluster using `kubeadm`.

Prerequisites

- Ubuntu OS (Xenial or later)
- `sudo` privileges
- Internet access
- t2.medium instance type or higher

AWS Setup

1. Ensure that all instances are in the same **Security Group**.
2. Expose port **6443** in the **Security Group** to allow worker nodes to join the cluster.
3. Expose port **22** in the **Security Group** to allows SSH access to manage the instance..

To do above setup, follow below provided steps

Step 1: Identify or Create a Security Group

1. **Log in to the AWS Management Console**:
    - Go to the **EC2 Dashboard**.
2. **Locate Security Groups**:
    - In the left menu under **Network & Security**, click on **Security Groups**.
3. **Create a New Security Group**:
    - Click on **Create Security Group**.
    - Provide the following details:
        - **Name**: (e.g., `Kubernetes-Cluster-SG`)
        - **Description**: A brief description for the security group (mandatory)

        - **VPC**: Select the appropriate VPC for your instances (default is acceptable)

4. **Add Rules to the Security Group**:
    - **Allow SSH Traffic (Port 22)**:
        - **Type**: SSH
        - **Port Range**: `22`
        - **Source**: `0.0.0.0/0` (Anywhere) or your specific IP
    - **Allow Kubernetes API Traffic (Port 6443)**:
        - **Type**: Custom TCP
        - **Port Range**: `6443`
        - **Source**: `0.0.0.0/0` (Anywhere) or specific IP ranges
5. **Save the Rules**:
    - Click on **Create Security Group** to save the settings.

Step 2: Select the Security Group While Creating Instances

- When launching EC2 instances:
    - Under **Configure Security Group**, select the existing security group (`Kubernetes-Cluster-SG`)

Note: Security group settings can be updated later as needed.

Execute on Both "Master" & "Worker" Nodes

1. **Disable Swap**: Required for Kubernetes to function correctly.

```
sudo swapoff -a
```

2. **Load Necessary Kernel Modules**: Required for Kubernetes networking.

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter
```

9. **Set Sysctl Parameters**: Helps with networking.

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF

sudo sysctl --system
lsmod | grep br_netfilter
lsmod | grep overlay
```

18. **Install Containerd**:

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo \"$VERSION_CODENAME\") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install -y containerd.io

containerd config default | sed -e 's/SystemdCgroup = false/SystemdCgroup = true/' -e 's/sandbox_image = "registry.k8s.io\/pause:3.6"/sandbox_image = "registry.k8s.io\/pause:3.9"/' | sudo tee /etc/containerd/config.toml

sudo systemctl restart containerd
sudo systemctl status containerd
```

33. **Install Kubernetes Components**:

```
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gpg

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list

sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

Execute ONLY on the "Master" Node

1. **Initialize the Cluster**:

```
sudo kubeadm init
```

2. **Set Up Local kubeconfig**:

```
3. mkdir -p "$HOME"/.kube
4. sudo cp -i /etc/kubernetes/admin.conf "$HOME"/.kube/config
   sudo chown "$(id -u)":"$(id -g)" "$HOME"/.kube/config
```

5. **Install a Network Plugin (Calico)**:

```
kubectl apply -f
https://raw.githubusercontent.com/projectcalico/calico/v3.26.0/manifests/ca
lico.yaml
```

6. **Generate Join Command**:

```
kubeadm token create --print-join-command
```

Copy this generated token for next command.

Execute on ALL of your Worker Nodes

1. Perform pre-flight checks:

```
sudo kubeadm reset pre-flight checks
```

2. Paste the join command you got from the master node and append `--v=5` at the end:

```
3. sudo kubeadm join <private-ip-of-control-plane>:6443 --token <token> --
   discovery-token-ca-cert-hash sha256:<hash> --cri-socket
   "unix:///run/containerd/containerd.sock" --v=5
```

   Note: When pasting the join command from the master node:

   i. Add `sudo` at the beginning of the command
   ii. Add `--v=5` at the end

   Example format:

```
sudo <paste-join-command-here> --v=5
```

Verify Cluster Connection

On Master Node:

```
kubectl get nodes
```

```
ubuntu@ip-172-31-17-101:~$ kubectl get nodes
NAME                STATUS   ROLES           AGE     VERSION
ip-172-31-17-101    Ready    control-plane   12m     v1.29.0
ip-172-31-17-255    Ready    <none>          8m27s   v1.29.0
ubuntu@ip-172-31-17-101:~$
```

Verify Container Status on Worker Node

```
ubuntu@ip-172-31-24-98:~$ sudo ctr -n k8s.io containers list
CONTAINER                                                           IMAGE                                    RUNTIME
090129d47ca503a7620d4e63b085788be7910df4730065d38d06eb07a1d612e1    docker.io/calico/cni:v3.26.0             io.containerd.runc.v2
0e9bbf31af163410bb90263f75edc8f6f64e6528bbb5b44a7e85b2b4c1b0db22    registry.k8s.io/pause:3.8                io.containerd.runc.v2
2d3f4abea7072c558deb18aa76cd874629678c6ac2ff568125c1768440f06b7a    registry.k8s.io/pause:3.8                io.containerd.runc.v2
3805e29d2d0a017587ef0c43e8324c010fdb8959005041c25ebc7dcccfa27014    docker.io/calico/cni:v3.26.0             io.containerd.runc.v2
4e8e925eb7d2390890f929fd73b9fde7c3dfe7f62fa7214cc14763bdec9ec34f    docker.io/library/nginx:latest           io.containerd.runc.v2
69dcf9921c6b796c797c1a516cbddb7e047b8e44be8ad1ce821ea5de80e897ce    registry.k8s.io/pause:3.8                io.containerd.runc.v2
7daa675a2190bd94af1933f3c4d6e2ae9a45638d7d4d0f613804b723582d9de6    registry.k8s.io/kube-proxy:v1.29.7       io.containerd.runc.v2
d9ea8bf85933ff48aa8801ccc4939977543da4e3dc106399f069b433c9cb72e2    docker.io/calico/node:v3.26.0            io.containerd.runc.v2
ebc883899e2c9e15e65441d1e495e5bef91d0ade430d6a86dc045e39cc0ef637    docker.io/calico/node:v3.26.0            io.containerd.runc.v2
ubuntu@ip-172-31-24-98:~$ sudo ctr -n k8s.io tasks list
TASK                                                                PID     STATUS
4e8e925eb7d2390890f929fd73b9fde7c3dfe7f62fa7214cc14763bdec9ec34f    4821    RUNNING
69dcf9921c6b796c797c1a516cbddb7e047b8e44be8ad1ce821ea5de80e897ce    3378    RUNNING
2d3f4abea7072c558deb18aa76cd874629678c6ac2ff568125c1768440f06b7a    3402    RUNNING
7daa675a2190bd94af1933f3c4d6e2ae9a45638d7d4d0f613804b723582d9de6    3435    RUNNING
ebc883899e2c9e15e65441d1e495e5bef91d0ade430d6a86dc045e39cc0ef637    3801    RUNNING
0e9bbf31af163410bb90263f75edc8f6f64e6528bbb5b44a7e85b2b4c1b0db22    4761    RUNNING
```